# PROJECT DEVELOPMENT REPORT

## ESP32 Bluetooth A2DP Audio Bridge

*Iterative Design from Cloud-Controlled Prototype to Standalone Audio Player*

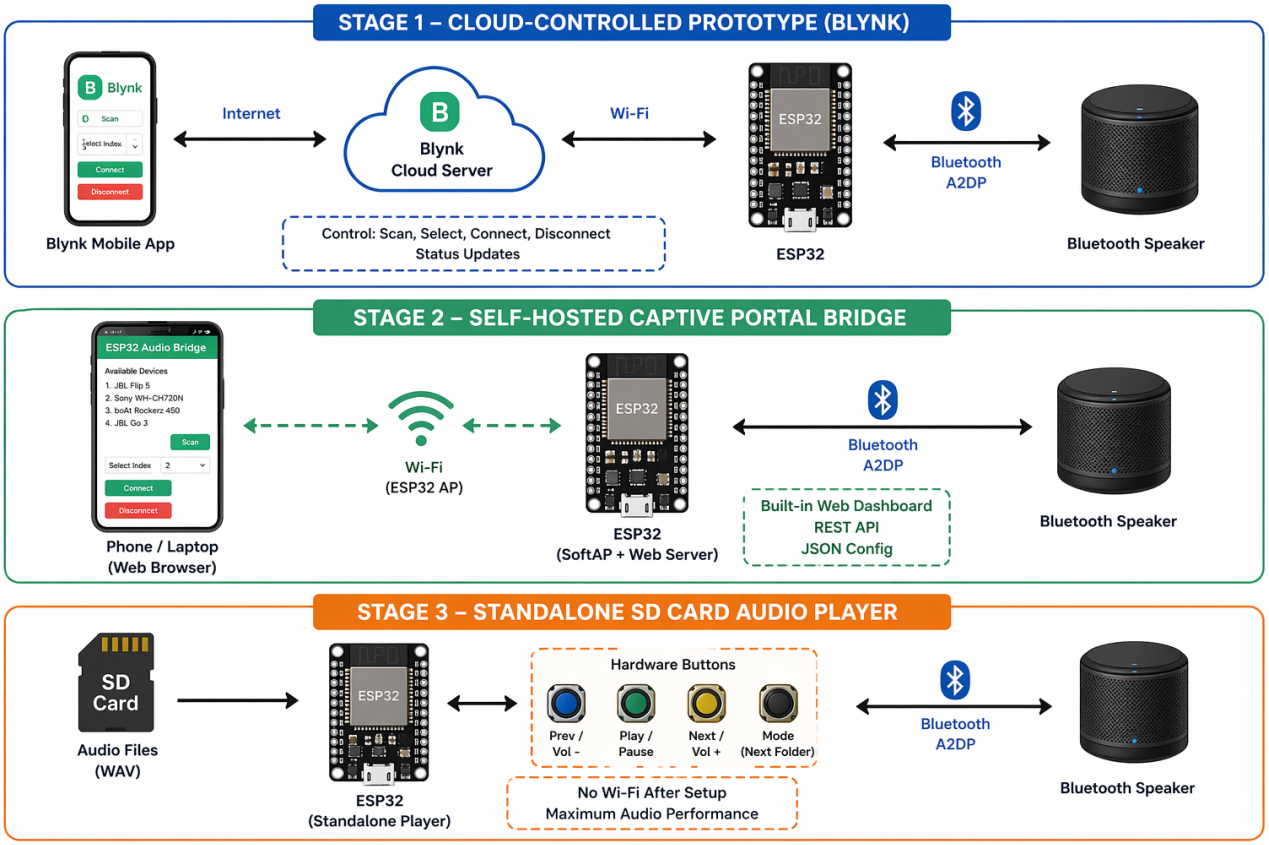
**Project Type:** Embedded Systems / IoT / Bluetooth Audio
**Hardware Platform:** Espressif ESP32 (Dual-Core, Classic BT + Wi-Fi)
**Development Stages:** 3 (Blynk Prototype -> Captive Portal -> Standalone SD Player)
**Primary Libraries:** BluetoothA2DPSource, BlynkSimpleEsp32, WebServer, ArduinoJson, SD
**Date:** April 2026

# 1. Introduction & Project Overview

**This report documents the complete iterative development journey of an ESP32-based Bluetooth A2DP Audio Bridge. The project's goal was to create a device that could wirelessly stream audio to any Bluetooth speaker — a common problem in environments where a legacy speaker has no modern wireless capability, or where a central audio source needs to be wire-free.**

**Rather than arriving at a final design immediately, the project progressed through three distinct engineering phases, each solving the limitations of the previous one. This iterative process is a hallmark of embedded systems development, where hardware constraints, wireless coexistence issues, and usability requirements are discovered only through real-world testing.**

**The three development stages were:**



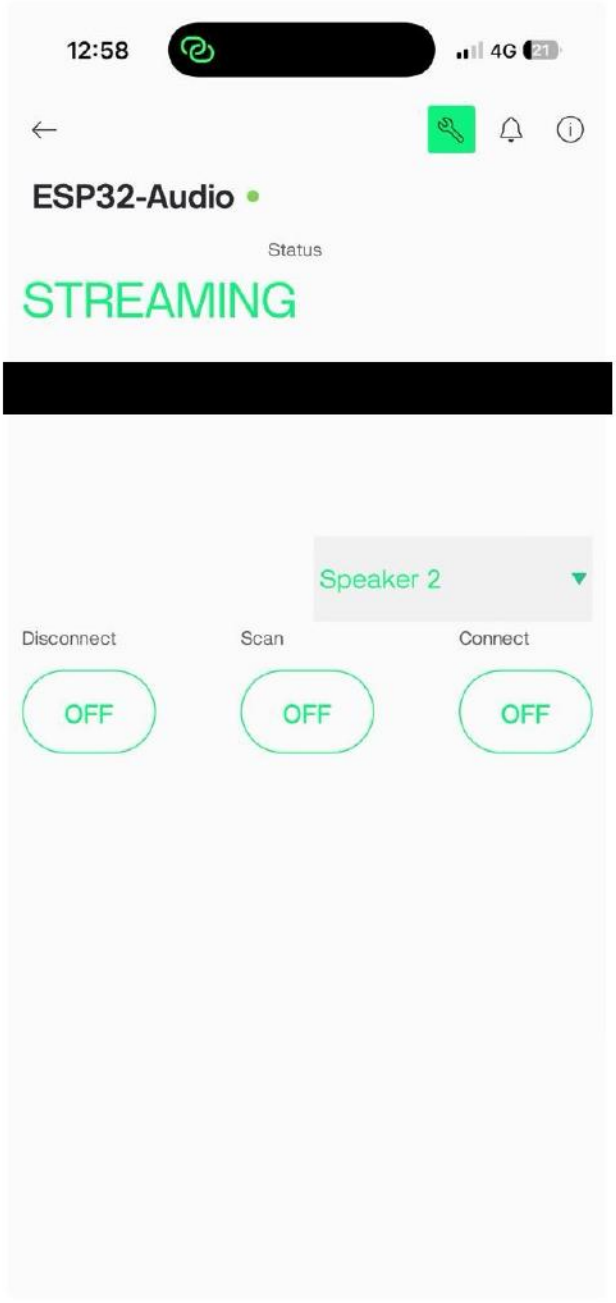
- Stage 1 — Cloud-Controlled Prototype (Blynk): An initial proof-of-concept using the Blynk IoT platform for remote control of Bluetooth scanning and connection over Wi-Fi.
- Stage 2 — Self-Hosted Captive Portal Bridge: A fully self-contained version with no dependency on external cloud services, using the ESP32 itself as a Wi-Fi Access Point with a built-in web dashboard.
- Stage 3 — Standalone SD Card Audio Player: The final, production-ready version that reads real audio files from an SD card, uses physical hardware buttons, and shuts down web services entirely after initial setup to maximise audio performance.

Each stage introduced significant new architecture, resolved critical bugs from the previous stage, and brought the project closer to a robust, real-world embedded product.

# 2. Stage 1 — Cloud-Controlled Prototype (Blynk IoT)

## 2.1 Concept and Architecture

**The first version of the project was a rapid proof-of-concept aimed at answering one question: can an ESP32 simultaneously maintain a Wi-Fi connection and perform Classic Bluetooth A2DP audio streaming? The chosen control platform was Blynk, a popular IoT dashboard service that allows developers to create mobile app interfaces with virtual pins mapped to microcontroller actions — all without writing any server-side code.**



**The architecture was simple: the ESP32 connected to an existing Wi-Fi network (a mobile hotspot), registered with Blynk's cloud servers, and exposed four virtual-pin controls — a scan button, an index selector, a connect button, and a disconnect button. A status string (V4) was continuously updated to reflect the current system state.**

## 2.2 Key Technical Decisions

- Used BluetoothA2DPSource library to act as a Bluetooth audio source (like a phone sending audio to a speaker).
- Implemented Classic BT inquiry using esp_bt_gap_register_callback and esp_bt_gap_start_discovery to discover nearby devices.
- Parsed Extended Inquiry Response (EIR) packets manually to extract human-readable device names, filtering out anonymous devices.
- Generated a 440 Hz sine wave tone as a placeholder audio stream to verify the A2DP pipeline was functioning.
- Disabled Wi-Fi sleep (WiFi.setSleep(false)) to prevent the antenna from being released during Bluetooth coexistence.

## 2.3 Core Code — Blynk Virtual Pin Handlers

**Scan trigger (V0) and connect trigger (V2):**

```
BLYNK_WRITE(V0) {  // Scan Button

  if (param.asInt() == 1) {

    startScanClassic(10); // 10 seconds scan

  }

}



BLYNK_WRITE(V2) {  // Connect Button

  if (param.asInt() == 1) {

    want_connect = true;

  }

}
```

**GAP callback for discovering and storing nearby Bluetooth devices:**

```
void gapCallback(esp_bt_gap_cb_event_t event, esp_bt_gap_cb_param_t *param) {
```

```
  if (event == ESP_BT_GAP_DISC_RES_EVT) {
    // Extract device name from EIR packet
    String name = extractNameFromEir(
      (uint8_t*)param->disc_res.prop[i].val,
      (uint8_t)param->disc_res.prop[i].len);
    // Store if new device with a valid name
    if (name.length() > 0 && dev_count < MAX_DEV) {
      memcpy(devices[dev_count].addr, bda, ESP_BD_ADDR_LEN);
      devices[dev_count].name = name;
      devices[dev_count].valid = true;
      dev_count++;
    }
  } else if (event == ESP_BT_GAP_DISC_STATE_CHANGED_EVT) {
    if (param->disc_st_chg.state == ESP_BT_GAP_DISCOVERY_STOPPED)
      scan_done = true;
  }
}
```

**Audio generation callback producing a 440 Hz test tone:**

```
int32_t get_sound_data(Frame *data, int32_t frameCount) {
  static float phase = 0.0f;
  const float freq = 440.0f;
  const float sample_rate = 44100.0f;
  if (frameCount > 128) frameCount = 128;
  for (int i = 0; i < frameCount; i++) {
    float v = sinf(phase) * 0.25f;
    int16_t s = (int16_t)(v * 32767);
```

```
    data[i].channel1 = s;

    data[i].channel2 = s;

    phase += 2.0f * PI * freq / sample_rate;

    if (phase > 2.0f * PI) phase -= 2.0f * PI;

  }

  return frameCount;

}
```

## 2.4 Problems Identified in Stage 1

**While Stage 1 successfully validated the concept, several critical limitations were identified during testing:**

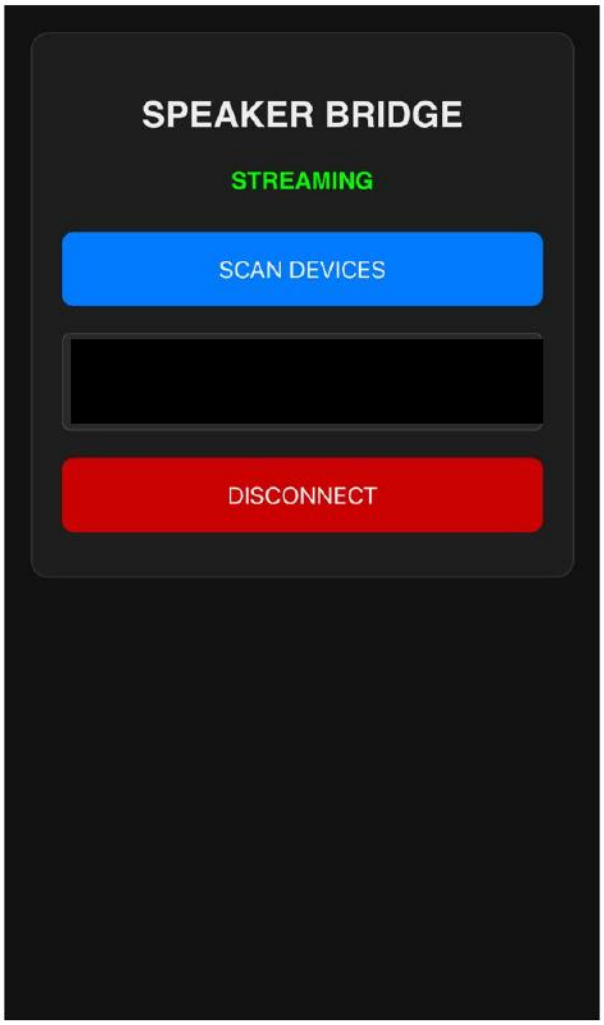
- Cloud Dependency: The entire control flow depended on Blynk's external servers. Any internet outage, hotspot unavailability, or Blynk service disruption would make the device completely uncontrollable.
- Wi-Fi / BT Coexistence Instability: The ESP32's single radio must time-share between Wi-Fi (for Blynk) and Classic Bluetooth (for A2DP). When actively streaming audio, the Blynk status updates became laggy and unreliable.
- No Real Audio: The 440 Hz sine wave was useful for pipeline verification but was not a deployable audio experience.
- Scalability: The Blynk free tier limited the number of virtual pins and data events, making feature expansion costly.

# 3. Stage 2 — Self-Hosted Captive Portal Bridge

## 3.1 Design Philosophy and Motivation

**The key insight from Stage 1 was that the dependency on external infrastructure was an architectural flaw, not just an inconvenience. A robust embedded device should operate independently of the cloud. Stage 2 was redesigned from the ground up to be completely self-hosted: the ESP32 would itself become a Wi-Fi Access Point, serve a web dashboard from its own flash memory, and handle all control logic locally.**

**A Captive Portal was chosen as the user interface paradigm. When any phone connects to the ESP32's Wi-Fi network, all DNS queries are hijacked to redirect the browser to the ESP32's local IP address — automatically presenting the dashboard without the user needing to know any URL. This is the same technique used by hotel or airport Wi-Fi login pages.**



## 3.2 New Components Introduced

- WebServer (port 80): Serves the dashboard HTML and handles REST API calls from the browser.
- DNSServer: Intercepts all DNS queries and responds with the ESP32's own IP, triggering the captive portal popup on most operating systems.
- ArduinoJson: Serialises device lists and status into clean JSON responses for the web UI.
- Embedded HTML/CSS/JS Dashboard: A complete single-page application stored as a PROGMEM string in flash, featuring a dark theme, animated status indicator, and dynamic device list.

## 3.3 System Architecture

**The setup sequence was carefully ordered to avoid resource conflicts between the Bluetooth stack and the Wi-Fi subsystem:**

```
void setup() {

  // Step 1: Release BLE memory — we only need Classic BT

  esp_bt_controller_mem_release(ESP_BT_MODE_BLE);



  // Step 2: Start A2DP Source — initialises the BT controller

  a2dp_source.set_auto_reconnect(false);

  a2dp_source.start(get_sound_data);

  esp_bt_gap_register_callback(gapCallback);



  // Step 3: Start Wi-Fi as an Access Point

  WiFi.mode(WIFI_AP);

  WiFi.softAP(ap_ssid, ap_pass); // 'ESP_Audio_Bridge' / '12345678'

  WiFi.setSleep(true); // Required for BT/WiFi coexistence in AP mode



  // Step 4: Start DNS hijacking for captive portal

  dnsServer.start(DNS_PORT, "*", WiFi.softAPIP());
```

```
  // Step 5: Register REST API routes and start web server
  server.begin();
}
```

## 3.4 Web Dashboard and REST API

**The dashboard polled a /status endpoint every 3 seconds to update the connection indicator. Discovered devices were fetched from /devices as a JSON array and rendered as clickable cards. Tapping a device called /connect with the target MAC address as a query parameter, triggering the A2DP connection entirely within the browser interaction.**

**Server-side connect handler:**

```
server.on("/connect", HTTP_POST, [](){
  String m = server.arg("mac");
  esp_bd_addr_t addr; int b[6];
  if (sscanf(m.c_str(), "%X:%X:%X:%X:%X:%X",
      &b[0],&b[1],&b[2],&b[3],&b[4],&b[5]) == 6) {
    for (int i=0; i<6; i++) addr[i] = (uint8_t)b[i];
    a2dp_source.connect_to(addr);
  }
  server.send(200, "application/json", "{\"ok\":true}");
});
```

**Status JSON response:**

```
server.on("/status", [](){
  StaticJsonDocument<128> doc;
  doc["scanning"] = (bool)scanning;
  doc["scan_done"] = (bool)scan_done;
```

```
  doc["connected"] = (bool)a2dp_source.is_connected();

  String o; serializeJson(doc, o);

  server.send(200, "application/json", o);

});
```

## 3.5 RAM Optimisation

**The ESP32 has approximately 320 KB of usable SRAM. Running both a BT stack and a Wi-Fi stack simultaneously is memory-intensive. The first and most important optimisation was releasing the BLE (Bluetooth Low Energy) controller memory at startup, since only Classic Bluetooth was required for A2DP:**

```
// Frees ~90 KB of IRAM/DRAM used by BLE controller

esp_bt_controller_mem_release(ESP_BT_MODE_BLE);
```

**The full HTML dashboard was stored in flash (PROGMEM) using the Arduino string literal macro, preventing it from consuming precious RAM. The JSON response buffers used StaticJsonDocument for small responses and DynamicJsonDocument only for the variable-length device list.**

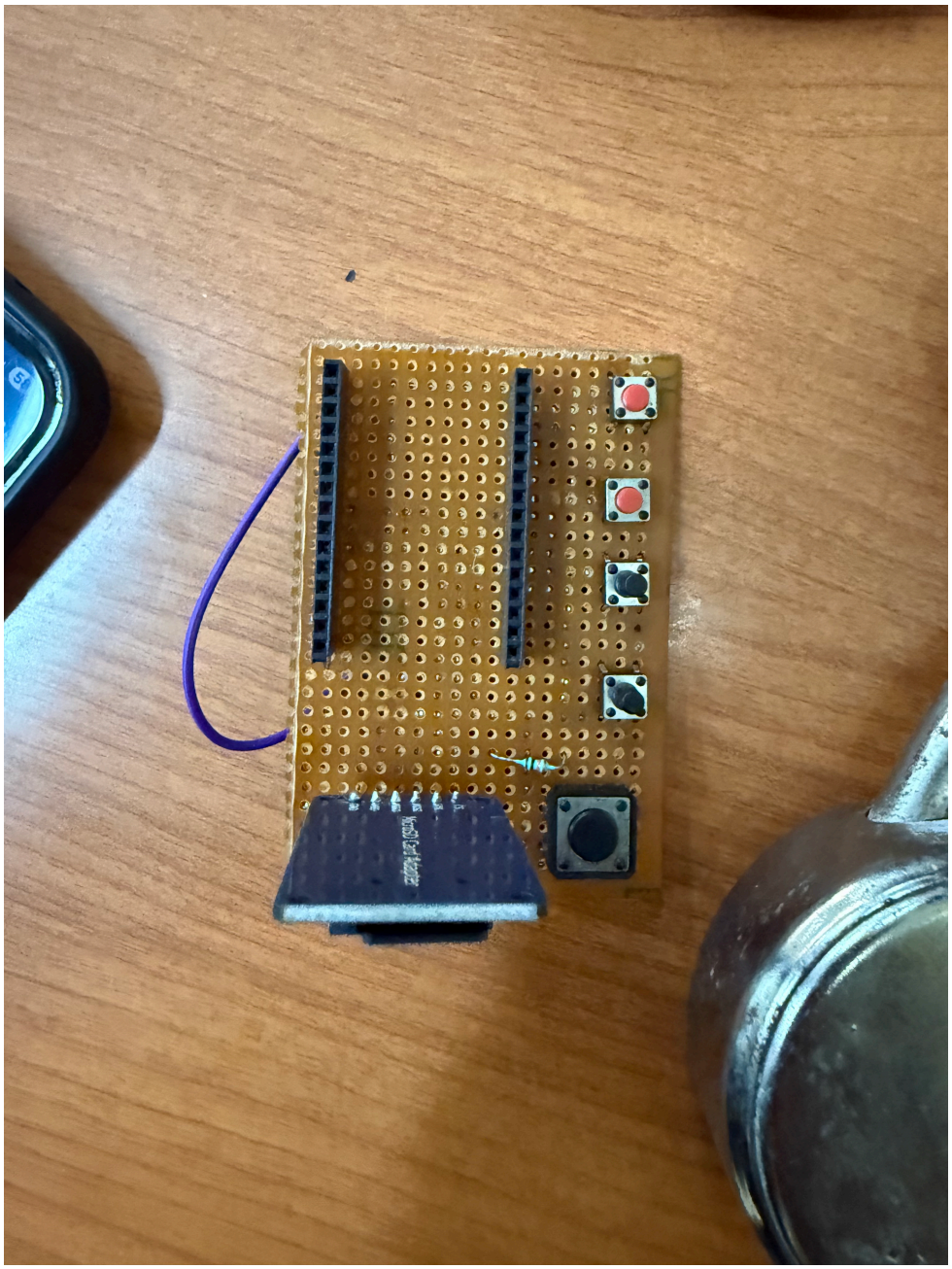
## 3.6 Limitations of Stage 2

- The sine wave audio was still a test tone, not real music.
- After connecting to the Bluetooth speaker, the ESP32 still ran the Wi-Fi Access Point and web server, consuming CPU cycles and antenna time-share that could otherwise be dedicated to audio.
- No physical controls existed — all interaction required a phone with Wi-Fi connectivity.
- The /disconnect button in the UI was a usability afterthought with no graceful teardown of the web connection.

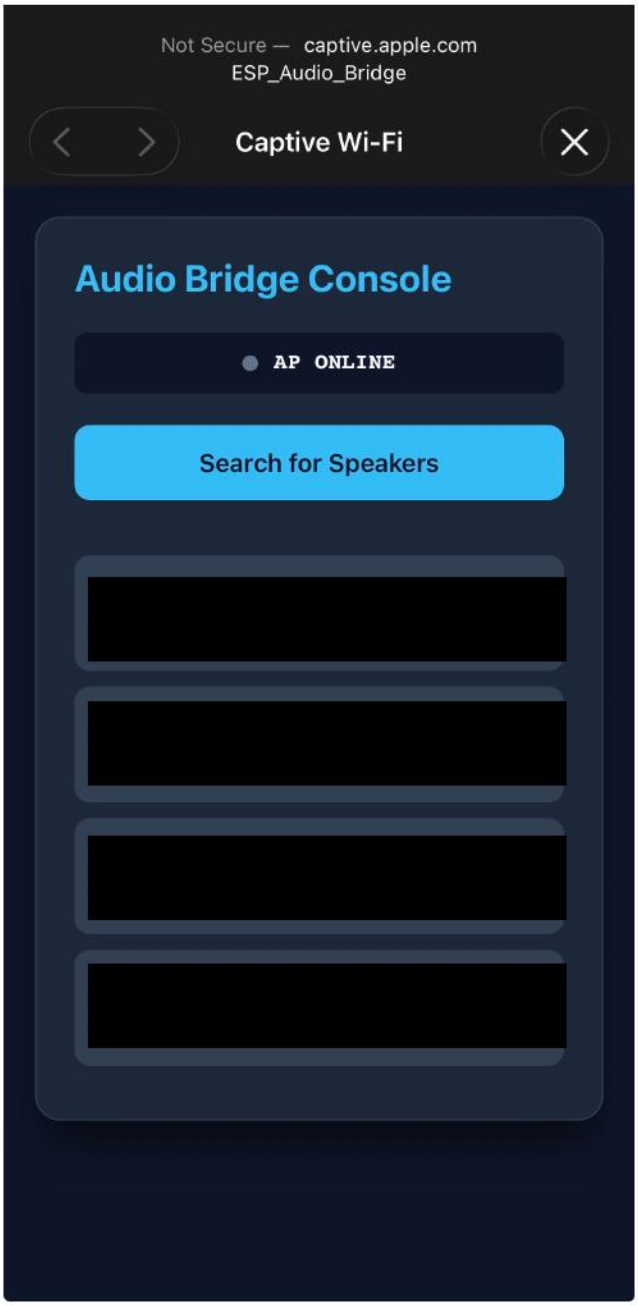
# 4. Stage 3 — Standalone SD Card Audio Player (Final Version)

## 4.1 Vision: A Truly Self-Contained Device

**Stage 3 represents the completion of the project's core vision. The goal was to build a device that requires zero ongoing infrastructure: no cloud, no phone app, no persistent Wi-Fi — just an ESP32, a microSD card loaded with WAV files, a Bluetooth speaker, and five physical push buttons.**



**The clever architectural innovation of Stage 3 is the Configuration Trap — a blocking loop at the end of setup() that holds the ESP32 in a web-enabled configuration mode until the user connects to the AP, opens the dashboard, and selects a Bluetooth speaker. Once the speaker is chosen, the web services are fully torn down, Wi-Fi is disabled, and the ESP32 dedicates its entire compute and radio resources to SD card streaming over Bluetooth.**



## 4.2 Hardware Additions

**Five GPIO pins were mapped to physical push buttons for hands-free playback control:**

```
#define SD_CS            5   // SD card chip select
#define BTN_PLAY_PAUSE   25  // Play / Pause toggle (INPUT_PULLUP)
#define BTN_NEXT         34  // Next track (INPUT, no internal pullup on ADC pin)
#define BTN_PREV         35  // Previous track (INPUT, no internal pullup on ADC pin)
#define BTN_VOL_DOWN     32  // Volume down (INPUT_PULLUP)
#define BTN_VOL_UP       33  // Volume up (INPUT_PULLUP)
```

**Note: GPIO 34 and 35 are input-only ADC pins on the ESP32 with no internal pull-up resistors, so they are configured with pinMode(pin, INPUT) and require external 10k pull-up resistors to 3.3V on the PCB.**

## 4.3 The Playlist System

**A compile-time playlist array maps track indices to file paths on the SD card. The project was designed for a specific geography education application, with WAV files named after the 28 states and 8 Union Territories of India:**

```
const char* playlist[] = {
  "/Uttar Pradesh.wav",   "/Rajasthan.wav",
  "/Bihar.wav",           "/Sikkim.wav",
  "/Himachal Pradesh.wav","/Punjab.wav",
  // ... 30 more entries ...
  "/Andaman and Nicobar.wav",
  "/Puduchery.wav"
};
```

```
const int NUM_TRACKS = sizeof(playlist) / sizeof(playlist[0]); // = 36
```

## 4.4 The SD Audio Pipeline — Mono-to-Stereo Fix

**The WAV files on the SD card were recorded as 16-bit mono PCM at 44100 Hz. The A2DP protocol, however, requires stereo audio (two channels). A naive implementation would simply send the same data to both channels, but this introduces a subtle memory corruption bug: the output Frame buffer has room for N stereo samples (N * 4 bytes), while reading N mono samples only fills N * 2 bytes — leaving the top half of the buffer unwritten.**

**The fix required reading only half the frame count's worth of bytes, then expanding the data backwards through the buffer — a technique that avoids a separate temporary buffer and operates safely in-place:**

```
int32_t get_sound_data(Frame *data, int32_t frameCount) {
  if (!isPlaying || !audioFile || !audioFile.available()
      || !a2dp_source.is_connected()) {
    memset(data, 0, frameCount * sizeof(Frame));
    return frameCount;
  }

  // Re-use the stereo buffer as a temporary mono read buffer
  int16_t *monoData = (int16_t *)data;
```

```
  // Read HALF the bytes (mono = 2 bytes/sample vs stereo = 4 bytes/sample)
  size_t bytesToRead = frameCount * 2;
  size_t bytesRead = audioFile.read((uint8_t*)monoData, bytesToRead);

  if (bytesRead == 0) { isPlaying = false; return 0; }
```

```
  int samplesRead = bytesRead / 2;



  // CRITICAL: Expand BACKWARDS to avoid overwriting unread samples

  for (int i = samplesRead - 1; i >= 0; i--) {

    int16_t sample = monoData[i];

    data[i].channel1 = sample;  // Left ear

    data[i].channel2 = sample;  // Right ear

  }

  return samplesRead;

}
```

**The WAV header (44 bytes) is skipped using audioFile.seek(44) before playback begins, avoiding the transmission of file metadata as audible noise.**

## 4.5 The Configuration Trap — Blocking Setup Pattern

**The most architecturally significant innovation of Stage 3 is how web services are handled. Rather than running permanently in the background, they exist only during the initial configuration phase. A flag variable speaker_selected is set to true by the /connect HTTP handler. The end of setup() then blocks in a tight loop, servicing DNS and HTTP requests, until this flag is raised:**

```
// --- THE CONFIGURATION TRAP ---

while (!speaker_selected) {

  dnsServer.processNextRequest();

  server.handleClient();

  delay(10);

}



// --- TEARDOWN SEQUENCE ---
```

```
Serial.println("[SYS] Speaker selected. Stopping web services...");

delay(1000); // Grace period for HTTP response to reach the browser



server.stop();

dnsServer.stop();

WiFi.mode(WIFI_OFF); // Completely disable Wi-Fi radio



Serial.println("[SYS] Dedicated to SD Audio Streaming.");
```

**After teardown, loop() never calls server.handleClient() or dnsServer.processNextRequest(), so the entire CPU budget is available for button scanning and the audio callbacks fired by the A2DP task.**

## 4.6 Button Handling with Debounce

**Physical buttons suffer from contact bounce — a single press can register dozens of transitions in under a millisecond. The debounce implementation tracks both the previous logic level and the last event timestamp, only registering an action when the button state changes AND sufficient time (250 ms) has elapsed:**

```
// PLAY/PAUSE — falling edge detection with 250ms debounce

if (currPlay == LOW && lastPlay == HIGH

    && (currentMillis - lastPlayTime > DEBOUNCE_TIME)) {

  if (isPlaying) {

    isPlaying = false;

    Serial.println("Paused");

  } else {

    if (a2dp_source.is_connected()) playTrack(currentTrack);

  }

  lastPlayTime = currentMillis;
```

```
}



// NEXT TRACK

if (currNext == LOW && lastNext == HIGH

    && (currentMillis - lastNextTime > DEBOUNCE_TIME)) {

  currentTrack = (currentTrack + 1) % NUM_TRACKS;

  if (a2dp_source.is_connected()) playTrack(currentTrack);

  lastNextTime = currentMillis;

}
```

**Volume is controlled via a2dp_source.set_volume(), an integer from 0 to 127, adjusted in steps of 10 per button press with clamping at the boundaries.**

## 4.7 Thread-Safe Track Switching

**The A2DP audio callback runs in a separate FreeRTOS task on Core 0, while button handling runs in loop() on Core 1. When the user requests a track change, a naive implementation would close and reopen the SD file while the audio task is mid-read — causing a crash. The fix is a two-step safety lock:**

```
void playTrack(int index) {

  // Step 1: Signal the BT audio task to stop reading immediately

  isPlaying = false;



  // Step 2: Wait one audio frame cycle (30ms) for safety

  delay(30);



  // Now safe to close and reopen the file

  if (audioFile) audioFile.close();
```

```
  audioFile = SD.open(playlist[index]);

  if (audioFile) {

    audioFile.seek(44); // Skip WAV header

    isPlaying = true;

  }

}
```

# 5. Comparative Analysis of All Three Stages

| Feature / Aspect | Stage 1 (Blynk) | Stage 2 (Captive Portal) | Stage 3 (Standalone SD) |
|---|---|---|---|
| Control Method | Blynk cloud (mobile app) | Browser (captive portal) | Physical buttons |
| Wi-Fi Mode | Station (existing network) | Access Point (self-hosted) | Disabled after setup |
| Internet Required | Yes (Blynk servers) | No | No |
| Audio Source | 440 Hz sine wave | 440 Hz sine wave | WAV files from SD card |
| BT Scan UI | Blynk virtual pins V10-V14 | Dynamic web UI cards | Web UI (setup only) |
| Physical Controls | None | None | 5 buttons (play, skip, vol) |
| RAM Efficiency | Moderate | Good (BLE released) | Excellent (Wi-Fi off after setup) |
| Production Ready | No | Partial | Yes |

# 6. Key Technical Challenges and Solutions

## 6.1 Wi-Fi and Bluetooth Radio Coexistence

**The ESP32 integrates both Wi-Fi and Classic Bluetooth on a single 2.4 GHz radio with a shared antenna. Both protocols must time-multiplex access to the antenna using an internal coexistence controller. Misconfiguration results in audio stuttering, connection drops, or failed BT inquiries.**

- Stage 1 fix: WiFi.setSleep(false) prevents the Wi-Fi stack from aggressively sleeping and refusing antenna hand-offs to BT.
- Stage 2 fix: WiFi.setSleep(true) in AP mode enables cooperative scheduling between the AP and BT stacks — counter-intuitively, sleep-enabled in AP mode is the correct coexistence setting.
- Stage 3 fix: The ultimate solution — disable Wi-Fi entirely after setup with WiFi.mode(WIFI_OFF), eliminating all coexistence contention.

## 6.2 EIR Name Parsing

**Bluetooth Classic device discovery returns raw binary Extended Inquiry Response (EIR) packets. Extracting a human-readable name requires manual parsing of the type-length-value (TLV) structure. Two type codes indicate device names: 0x09 (Complete Local Name) and 0x08 (Shortened Local Name). The parser iterates through fields, skipping unknown types, and returns the first matching name string found.**

**Devices that advertise no name (e.g., headsets in pairing mode before a name is set) are silently filtered out to keep the device list clean.**

## 6.3 In-Place Mono-to-Stereo Expansion

**As detailed in Section 4.4, the WAV files were mono but the A2DP protocol requires stereo. The challenge was to perform this conversion without allocating an additional heap buffer (which would risk fragmentation at runtime). The backwards-expansion technique reuses the output Frame buffer as both the read target and the write target, guaranteed safe because the write index is always equal to or ahead of the read index during the backwards pass.**

## 6.4 FreeRTOS Task Safety for File I/O

**The ESP32 Arduino framework runs the user's loop() function on Core 1 and the Bluetooth A2DP callback on Core 0, both as FreeRTOS tasks. Accessing the SD card from both cores simultaneously without synchronisation is undefined behaviour. The solution — setting isPlaying = false and waiting 30ms before touching the file — is a pragmatic spinlock-free approach appropriate for the non-real-time requirements of playback control.**

## 7. Conclusion

**This project demonstrates a complete embedded systems design journey — from a rapid cloud-dependent prototype to a fully self-contained, production-quality device. Each stage built meaningfully on the last, driven by concrete technical limitations discovered during testing rather than theoretical concerns.**

**Stage 1 proved the feasibility of simultaneous Wi-Fi + Classic BT operation and established the core BT scanning and A2DP pipeline. Stage 2 eliminated cloud dependency, introduced a professional captive portal UI, and optimised RAM by releasing unused BLE memory. Stage 3 delivered the complete product vision: real audio playback from an SD card, physical button controls, and a unique configuration-then-teardown architecture that achieves maximum audio performance by completely disabling Wi-Fi after the initial speaker selection.**

**The final Stage 3 firmware is a mature, robust piece of embedded software that addresses coexistence, thread safety, memory efficiency, and user experience simultaneously — qualities that emerge only through iterative development and honest problem diagnosis at each stage.**

# Appendix A: Complete Source Code (Stage 3)

## A.1 Final Firmware (Standalone SD Audio Player)

**The following section contains the final, compilable source code for the Stage 3 iteration of the ESP32 Audio Bridge. This firmware was compiled using the Arduino IDE with the official ESP32 Board Manager. External dependencies required for compilation include the `BluetoothA2DPSource` library for audio streaming, as well as the standard `SD`, `WebServer`, and `DNSServer` libraries .**

```
#include <WiFi.h>

#include <WebServer.h>

#include <DNSServer.h>

#include <ArduinoJson.h>

#include <SD.h>

#include <SPI.h>

#include "BluetoothA2DPSource.h"

#include "esp_bt_main.h"

#include "esp_gap_bt_api.h"

#include "esp_bt_device.h"


// --- PIN DEFINITIONS ---

#define SD_CS            5

#define BTN_PLAY_PAUSE   25

#define BTN_NEXT         34

#define BTN_PREV         35

#define BTN_VOL_DOWN     32

#define BTN_VOL_UP       33


// --- SETTINGS ---

const char* ap_ssid = "ESP_Audio_Bridge";
```

```
const char* ap_pass = "12345678";

const byte DNS_PORT = 53;



BluetoothA2DPSource a2dp_source;

WebServer server(80);

DNSServer dnsServer;

File audioFile;



// --- AUDIO PLAYLIST ---

const char* playlist[] = {"/Uttar Pradesh.wav", "/Rajasthan.wav", "/Bihar.wav", "/
Sikkim.wav",

    "/Himachal Pradesh.wav", "/Punjab.wav", "/Uttarakhand.wav", "/Haryana.wav",

    "/Tripura.wav", "/Meghalaya.wav", "/Assam.wav", "/West Bengal.wav",

    "/Arunachal Pradesh.wav", "/Nagaland.wav", "/Manipur.wav", "/Mizoram.wav",

    "/Jharkhand.wav", "/Chattisgarh.wav", "/Madhya Pradesh.wav", "/Gujarat.wav",

    "/Odisha.wav", "/Maharashtra.wav", "/Telangana.wav", "/Andhra Pradesh.wav",

    "/Goa.wav", "/Karnataka.wav", "/Kerala.wav", "/Tamil Nadu.wav",

    "/Ladakh.wav", "/Jammu and Kashmir.wav", "/Chandigarh.wav", "/Delhi.wav",

    "/Lakshwadeep.wav", "/Andaman and Nicobar.wav",

    "/Dadar and Nagar Haveli and Daman and Diu.wav", "/Puduchery.wav"};

const int NUM_TRACKS = sizeof(playlist) / sizeof(playlist[0]);

int currentTrack = 0;

volatile bool isPlaying = false;

int currentVolume = 60;



// --- GLOBALS ---

static const int MAX_DEV = 15;

struct BtDev { esp_bd_addr_t addr; String name; bool valid; };
```

```
BtDev devices[MAX_DEV];
volatile int dev_count = 0;
volatile bool scanning = false;
volatile bool scan_done = false;

// Kill Switch Flag
volatile bool speaker_selected = false;

// --- BUTTON DEBOUNCE GLOBALS ---
const unsigned long DEBOUNCE_TIME = 250;
unsigned long lastPlayTime = 0;
unsigned long lastNextTime = 0;
unsigned long lastPrevTime = 0;
unsigned long lastVolDTime = 0;
unsigned long lastVolUTime = 0;

bool lastPlay = HIGH;
bool lastNext = HIGH;
bool lastPrev = HIGH;
bool lastVolD = HIGH;
bool lastVolU = HIGH;

// --- MODERN UI (DASHBOARD) ---
const char INDEX_HTML[] PROGMEM = R"=====(
<!DOCTYPE html><html><head><title>Audio Bridge</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
```

```
  :root { --bg: #0f172a; --card: #1e293b; --accent: #38bdf8; --danger: #ef4444; --text:
#f8fafc; }

  body { font-family: system-ui, -apple-system, sans-serif; background: var(--bg); color:
var(--text); margin: 0; padding: 20px; }

  .container { max-width: 500px; margin: auto; }

  .card { background: var(--card); padding: 24px; border-radius: 16px; border: 1px solid
#334155; box-shadow: 0 20px 25px -5px rgba(0,0,0,0.5); }

  h2 { margin: 0 0 20px 0; color: var(--accent); font-size: 1.5rem; }

  .status-box { display: flex; align-items: center; justify-content: center; background:
#0f172a; padding: 12px; border-radius: 8px; margin-bottom: 20px; font-weight: bold; font-
family: monospace; font-size: 0.9rem; text-align: center; }

  .dot { height: 10px; width: 10px; border-radius: 50%; display: inline-block; margin-
right: 10px; background: #64748b; }

  .dot-online { background-color: #22c55e; box-shadow: 0 0 8px #22c55e; }

  .dot-pulse { background-color: var(--accent); animation: pulse 1.5s infinite; }

  @keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.3; } }

  .btn { width: 100%; padding: 14px; border: none; border-radius: 10px; font-weight: 600;
cursor: pointer; transition: 0.2s; font-size: 1rem; }

  .btn-primary { background: var(--accent); color: #0f172a; margin-bottom: 15px; }

  .btn-primary:active { transform: scale(0.98); }

  .btn-danger { background: transparent; color: var(--danger); border: 1px solid var(--
danger); }

  #list { margin-top: 20px; text-align: left; }

  .dev-item { background: #334155; padding: 16px; border-radius: 10px; margin-bottom:
10px; cursor: pointer; border: 1px solid transparent; }

  .dev-item:active { background: #475569; border-color: var(--accent); }

  .dev-name { font-weight: 600; display: block; color: #fff; }

  .dev-mac { font-size: 11px; color: #94a3b8; font-family: monospace; }

</style></head>

<body>

  <div class="container">

    <div class="card">
```

```
      <h2>Audio Bridge Console</h2>

      <div class="status-box">

        <span id="dot" class="dot"></span>

        <span id="st">INITIALIZING</span>

      </div>

      <button class="btn btn-primary" onclick="cmd('/scan')">Search for Speakers</button>

      <div id="list"></div>

    </div>

  </div>

<script>

  let pollingInterval;

  function cmd(p){

    fetch(p,{method:'POST'});

    if(p==='/scan'){

      document.getElementById('st').innerText = "SCANNING...";

      document.getElementById('dot').className = "dot dot-pulse";

      document.getElementById('list').innerHTML = "<p style='color:#94a3b8;text-
align:center'>Detecting Bluetooth signals...</p>";

    }

  }

  function connect(mac, name){

    document.getElementById('st').innerHTML = "CONNECTING...<br>UI WILL NOW GO OFFLINE";

    document.getElementById('dot').className = "dot dot-online";

    clearInterval(pollingInterval);

    fetch('/connect?mac='+mac,{method:'POST'});

  }



  pollingInterval = setInterval(()=>{
```

```
    fetch('/status').then(r=>r.json()).then(d=>{

      const st = document.getElementById('st');

      const dot = document.getElementById('dot');

      if(!d.scanning){

        st.innerText = d.connected ? "STREAMING ACTIVE" : "AP ONLINE";

        dot.className = d.connected ? "dot dot-online" : "dot";

      }

      if(d.scan_done) {

        fetch('/devices').then(r=>r.json()).then(devs=>{

          let h=''; devs.forEach(v=>{ h+=`<div class='dev-item' onclick="connect('${v.a}','${v.n}')"><span class='dev-name'>${v.n}</span><span class='dev-mac'>${v.a}</span></div>`; });

          document.getElementById('list').innerHTML = h;

        });

      }

    }).catch(e => {

       console.log("Server unreachable. Streaming mode active.");

    });

  },3000);

</script></body></html>

)=====";



// --- BT LOGIC HELPERS ---

String bdAddrToString(const esp_bd_addr_t bda) {

  char s[18];

  sprintf(s, "%02X:%02X:%02X:%02X:%02X:%02X", bda[0], bda[1], bda[2], bda[3], bda[4], bda[5]);

  return String(s);

}
```

```
String extractNameFromEir(const uint8_t *eir, uint8_t eir_len) {
  if (!eir || eir_len == 0) return "";
  int i = 0;
  while (i < eir_len) {
    uint8_t field_len = eir[i];
    if (field_len == 0) break;
    uint8_t type = eir[i + 1];
    if (type == ESP_BT_EIR_TYPE_CMPL_LOCAL_NAME || type ==
ESP_BT_EIR_TYPE_SHORT_LOCAL_NAME) {
      int name_len = field_len - 1;
      if (name_len > 0) {
        String nm = "";
        for (int k = 0; k < name_len; k++) nm += (char)eir[i + 2 + k];
        nm.trim(); return nm;
      }
    }
    i += field_len + 1;
  }
  return "";
}


void gapCallback(esp_bt_gap_cb_event_t event, esp_bt_gap_cb_param_t *param) {
  if (event == ESP_BT_GAP_DISC_RES_EVT) {
    esp_bd_addr_t bda; memcpy(bda, param->disc_res.bda, 6);
    String name = "";
    for (int i = 0; i < param->disc_res.num_prop; i++) {
      if (param->disc_res.prop[i].type == ESP_BT_GAP_DEV_PROP_EIR)
```

```
        name = extractNameFromEir((uint8_t*)param->disc_res.prop[i].val, (uint8_t)param->disc_res.prop[i].len);
    }
    if (name.length() == 0) return;
    for (int k = 0; k < dev_count; k++) {
      if (devices[k].valid && memcmp(devices[k].addr, bda, 6) == 0) return;
    }
    if (dev_count < MAX_DEV) {
      memcpy(devices[dev_count].addr, bda, 6);
      devices[dev_count].name = name;
      devices[dev_count].valid = true;
      Serial.printf("[BT] Found: %s\n", name.c_str());
      dev_count++;
    }
  } else if (event == ESP_BT_GAP_DISC_STATE_CHANGED_EVT) {
    if (param->disc_st_chg.state == ESP_BT_GAP_DISCOVERY_STOPPED) {
      Serial.println("[BT] Scan Finished.");
      scanning = false; scan_done = true;
    }
  }
}

// --- AUDIO FUNCTIONS ---
void playTrack(int index) {
    // 1. SAFETY LOCK: Tell the BT task to stop reading immediately
    isPlaying = false;

    // 2. Wait 30ms to ensure the BT task finishes its current read cycle safely
```

```
    delay(30);


    if (audioFile) {
        audioFile.close();
    }


    audioFile = SD.open(playlist[index]);
    if (audioFile) {
        audioFile.seek(44); // Skip header
        isPlaying = true;
        Serial.printf("▶️ Playing: %s\n", playlist[index]);
    } else {
        Serial.printf("❌ Failed to open: %s\n", playlist[index]);
        isPlaying = false;
    }
}


// --- STANDARD STEREO AUDIO DATA (Reverted) ---
// --- MONO-TO-STEREO FIX (Memory Safe) ---
int32_t get_sound_data(Frame *data, int32_t frameCount) {
  if (!isPlaying || !audioFile || !audioFile.available() || !a2dp_source.is_connected())
{
    memset(data, 0, frameCount * sizeof(Frame));
    return frameCount;
  }


  // Treat the original stereo buffer as a temporary mono buffer
```

```
  int16_t *monoData = (int16_t *)data;

  // Read half the amount of data (because Mono is exactly half the size of Stereo)
  size_t bytesToRead = frameCount * 2;
  size_t bytesRead = audioFile.read((uint8_t*)monoData, bytesToRead);

  if (bytesRead == 0) {
    isPlaying = false;
    Serial.println("⏹️ Audio finished.");
    return 0;
  }

  int samplesRead = bytesRead / 2;

  // Expand backwards to safely fill the stereo buffer without overwriting memory
  // This duplicates the Mono audio into both the Left and Right ears perfectly
  for (int i = samplesRead - 1; i >= 0; i--) {
    int16_t sample = monoData[i];
    data[i].channel1 = sample;
    data[i].channel2 = sample;
  }

  return samplesRead;
}

void setup() {
  Serial.begin(115200);
```

```
delay(1000);
Serial.println("\n--- STARTING CAPTIVE PORTAL AUDIO BRIDGE ---");

// 1. Initialize Buttons
pinMode(BTN_PLAY_PAUSE, INPUT_PULLUP);
pinMode(BTN_VOL_DOWN, INPUT_PULLUP);
pinMode(BTN_VOL_UP, INPUT_PULLUP);
pinMode(BTN_PREV, INPUT);
pinMode(BTN_NEXT, INPUT);

// Initialize button history with actual physical state
lastPlay = digitalRead(BTN_PLAY_PAUSE);
lastNext = digitalRead(BTN_NEXT);
lastPrev = digitalRead(BTN_PREV);
lastVolD = digitalRead(BTN_VOL_DOWN);
lastVolU = digitalRead(BTN_VOL_UP);

// 2. Initialize SD Card
if (!SD.begin(SD_CS, SPI, 4000000)) {
  Serial.println("❌ [ERROR] SD Mount Failed!");
} else {
  Serial.println("✅ [SYS] SD Card Mounted.");
}

// 3. RAM Optimization & BT Init
esp_bt_controller_mem_release(ESP_BT_MODE_BLE);
a2dp_source.set_auto_reconnect(false);
```

```
a2dp_source.start(get_sound_data);

a2dp_source.set_volume(currentVolume);

esp_bt_gap_register_callback(gapCallback);

Serial.println("[SYS] Bluetooth Stack Active.");



// 4. Initialize WiFi Access Point

WiFi.mode(WIFI_AP);

WiFi.softAP(ap_ssid, ap_pass);

Serial.print("[WIFI] AP Ready: "); Serial.println(WiFi.softAPIP());



// 5. Initialize DNS Redirect

dnsServer.start(DNS_PORT, "*", WiFi.softAPIP());



server.on("/", [](){ server.send_P(200, "text/html", INDEX_HTML); });

server.onNotFound([](){ server.send_P(200, "text/html", INDEX_HTML); });



server.on("/status", [](){

  StaticJsonDocument<128> doc;

  doc["scanning"] = (bool)scanning;

  doc["scan_done"] = (bool)scan_done;

  doc["connected"] = (bool)a2dp_source.is_connected();

  String o; serializeJson(doc, o); server.send(200, "application/json", o);

});



server.on("/devices", [](){

  DynamicJsonDocument doc(2048);

  JsonArray arr = doc.to<JsonArray>();
```

```
    for(int i=0; i<dev_count; i++){
      JsonObject obj = arr.add<JsonObject>();
      obj["n"] = devices[i].name;
      obj["a"] = bdAddrToString(devices[i].addr);
    }
    String o; serializeJson(doc, o); server.send(200, "application/json", o);
  });

  server.on("/scan", HTTP_POST, [](){
    if (a2dp_source.is_connected()) a2dp_source.disconnect();
    dev_count = 0; scanning = true; scan_done = false;
    esp_bt_gap_start_discovery(ESP_BT_INQ_MODE_GENERAL_INQUIRY, 10, 0);
    server.send(200, "application/json", "{\"ok\":true}");
  });

  server.on("/connect", HTTP_POST, [](){
    String m = server.arg("mac");
    esp_bd_addr_t addr; int b[6];
    if (sscanf(m.c_str(), "%X:%X:%X:%X:%X:%X", &b[0], &b[1], &b[2], &b[3], &b[4], &b[5])
== 6) {
      for (int i=0; i<6; i++) addr[i] = (uint8_t)b[i];
      Serial.printf("[WEB] Linking to MAC: %s\n", m.c_str());
      a2dp_source.connect_to(addr);
    }
    server.send(200, "application/json", "{\"ok\":true}");

    speaker_selected = true;
  });
```

```
  server.begin();
  Serial.println("[SYS] Web Server Operational. Waiting for user setup...");

  // --- 6. THE CONFIGURATION TRAP ---
  while (!speaker_selected) {
    dnsServer.processNextRequest();
    server.handleClient();
    delay(10);
  }

  // --- 7. TEARDOWN SEQUENCE ---
  Serial.println("[SYS] Speaker selected. Stopping web services...");
  delay(1000);

  server.stop();
  dnsServer.stop();
  WiFi.mode(WIFI_OFF);

  Serial.println("[SYS] Web services disabled. ESP32 is now dedicated to SD Audio Streaming.");
}

// --- MAIN LOOP ---
void loop() {

    // Read Current Button States
    bool currPlay = digitalRead(BTN_PLAY_PAUSE);
```

```
    bool currNext = digitalRead(BTN_NEXT);

    bool currPrev = digitalRead(BTN_PREV);

    bool currVolD = digitalRead(BTN_VOL_DOWN);

    bool currVolU = digitalRead(BTN_VOL_UP);



    unsigned long currentMillis = millis();



    // PLAY/PAUSE (D25)

    if (currPlay == LOW && lastPlay == HIGH && (currentMillis - lastPlayTime >
DEBOUNCE_TIME)) {

        if (isPlaying) {

            isPlaying = false;

            Serial.println("⏸️ Paused");

        } else {

            if (a2dp_source.is_connected()) {

                playTrack(currentTrack);

            } else {

                Serial.println("⏳ Waiting for Bluetooth...");

            }

        }

        lastPlayTime = currentMillis;

    }



    // NEXT TRACK (D34)

    if (currNext == LOW && lastNext == HIGH && (currentMillis - lastNextTime >
DEBOUNCE_TIME)) {

        currentTrack = (currentTrack + 1) % NUM_TRACKS;

        if (a2dp_source.is_connected()) playTrack(currentTrack);
```

```
        lastNextTime = currentMillis;

    }



    // PREVIOUS TRACK (D35)

    if (currPrev == LOW && lastPrev == HIGH && (currentMillis - lastPrevTime >
DEBOUNCE_TIME)) {

        currentTrack = (currentTrack - 1 + NUM_TRACKS) % NUM_TRACKS;

        if (a2dp_source.is_connected()) playTrack(currentTrack);

        lastPrevTime = currentMillis;

    }



    // VOLUME DOWN (D32)

    if (currVolD == LOW && lastVolD == HIGH && (currentMillis - lastVolDTime >
DEBOUNCE_TIME)) {

        currentVolume = max(0, currentVolume - 10);

        a2dp_source.set_volume(currentVolume);

        Serial.printf("🔉 Volume: %d\n", currentVolume);

        lastVolDTime = currentMillis;

    }



    // VOLUME UP (D33)

    if (currVolU == LOW && lastVolU == HIGH && (currentMillis - lastVolUTime >
DEBOUNCE_TIME)) {

        currentVolume = min(127, currentVolume + 10);

        a2dp_source.set_volume(currentVolume);

        Serial.printf("🔊 Volume: %d\n", currentVolume);

        lastVolUTime = currentMillis;

    }
```

```
    // Update Last States
    lastPlay = currPlay;
    lastNext = currNext;
    lastPrev = currPrev;
    lastVolD = currVolD;
    lastVolU = currVolU;
}
```

*End of Report — ESP32 Bluetooth A2DP Audio Bridge Development*